

Tutorial: Pogotrack Data Analysis

Purpose. This document explains how to use Pogotrack to analyse Pogobot swarm experiments. It is written as a practical guideline rather than as complete library documentation, with step-by-step examples that should make the analysis reproducible for the whole team.

Maintainer. Keivan Amini

Date. 22 September 2026

Contact. For questions or improvements, contact amini@isir.upmc.fr or keivan.amini@espci.fr or open an issue or pull request in the repository.

Contents

1	Introduction	1
2	Installation	3
3	Repository and experiment structure	4
4	Core analysis: one experiment	5
5	Important configuration parameters	7
6	Relevant modules and classes	8
7	Run all analysis all at once	9
8	Understanding the merged data and slides	11
9	Troubleshooting checklist	11
10	Final remarks	12

1 Introduction

1.1 Purpose and scope

Pogotrack is a research tool for reconstructing the trajectories of Pogobot swarms from video. It detects the position and orientation of each robot, links detections through time, and exports a tabular dataset that can be combined with the variables recorded by the robots, such as controller scores and weights.

This document is a practical guide rather than a complete software reference. It is written for a member of the team who has the raw files for an experiment and wants to reproduce the analysis from start to finish. The examples assume that commands are executed from the root of the Pogotrack repository.

The current maintainer is Keivan Amini, with contributions from the experimental and robotics teams, including J  r  my Fersula. Questions, corrections, and improvements are welcome.

1.2 The analysis pipeline

Pogotrack supports circular and rectangular arenas. The phototaxis workflow has currently been tested at ESPCI and uses separate processing for the dark and illuminated parts of the arena. Most experiment-specific settings, such as the arena geometry, number of robots, thresholds, and calibration, belong in YAML configuration files. Editing the detector code should not be necessary for a standard experiment.

A complete experiment normally provides the following files:

- a dynamics video, for example `alpha10-4_T100_v2.mp4`;
- an RGB-ID video, conventionally named by adding `id_` before the dynamics-video name;
- `bkg.bmp`, the background image of the arena;
- a Feather file containing the internal variables recorded by the robots, usually named after the experiment.

The workflow has four stages:

1. **Dynamics tracking.** The dynamics video is converted into one row per robot and frame, containing position, orientation, time, and a temporary TrackPy identity.
2. **RGB-ID tracking.** The RGB-ID video is analysed to measure the red, green, and blue LED intensities for each robot during the flashing sequence.
3. **Decoding and merging.** The RGB measurements are decoded into robot IDs and joined with the dynamics CSV and the controller data from the Feather file.
4. **Summary figures.** The merged dataset is used to produce PNG and PDF summary slides containing controller, trajectory, light-occupancy, and space-time plots.

The first two stages are independent video-processing jobs. The third stage connects their outputs, and the fourth stage is optional but useful for a quick quality check.



Figure 1: Pogotrack logo.

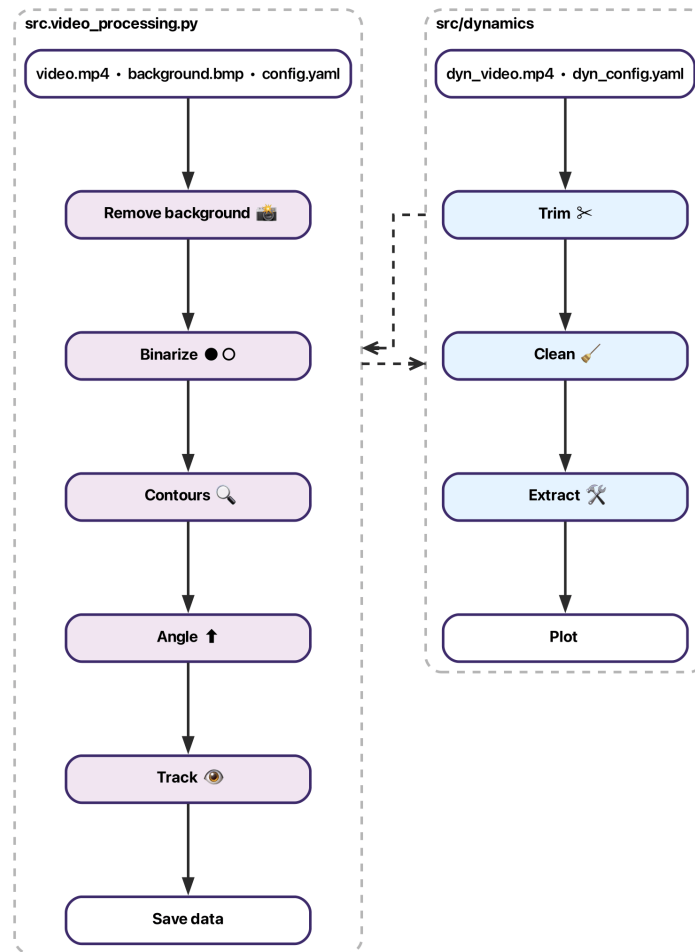


Figure 2: Left panel: overview of the Pogotrack dynamics analysis pipeline.

2 Installation

2.1 System requirements

Pogotrack supports CPython 3.11–3.13; Python 3.13 is recommended on Apple Silicon macOS. The versions used by the project are listed in `requirements.txt`. The workflow also needs the `ffmpeg` command-line program. The optional `tmux` launcher needs `tmux` as well.

On macOS with Homebrew, a typical installation is:

```

brew install python@3.13 ffmpeg tmux
cd /path/to/pogotrack
export DYLD_LIBRARY_PATH="$(brew --prefix expat)/lib${DYLD_LIBRARY_PATH:+:$DYLD_LIBRARY_PATH}"
python3.13 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
python -m pip install -r requirements.txt
  
```

On some Homebrew installations, the `DYLD_LIBRARY_PATH` line is needed because `ensurepip` selects the system `Expatriate` library. After activating the environment, check the interpreter before running the

pipeline:

```
python --version
which python
```

The first command should report Python 3.11 or newer. The second should point inside `.venv/bin/`.

Attention. Pogotrack rejects Python versions older than 3.11. Always activate the project virtual environment first.

Once the environment is active, all examples in this guide use `python`. If the environment is not active, replace `python` with the full path to the project interpreter, for example `.venv/bin/python`.

3 Repository and experiment structure

The most important repository directories are:

Path	Role
<code>main.py</code>	Command-line entry point for standard tracking, phototaxis tracking, RGB-ID tracking, and optional parallel frame processing.
<code>src/</code>	Core computer-vision, tracking, plotting, and dynamics modules.
<code>config/</code>	YAML configuration files. <code>default.yaml</code> is the principal tracking configuration.
<code>data/</code>	Experimental input folders containing videos, backgrounds, and robot measurements.
<code>results/</code>	Generated tracking CSV files, merged datasets, logs, plots, and slides.
<code>postprocessing/</code>	Batch analysis, RGB-ID decoding, automatic merging, and experiment-slide generation.
<code>stl/</code>	Pogobot exoskeleton files used by the experimental setup.
<code>requirements.txt</code>	Pinned Python dependencies for the supported runtime.

For one experiment, the recommended input layout is:

```
data/
+-- 18-09-26/
  +-- alpha10-4_T100_v2.mp4      # dynamics video
  +-- id_alpha10-4_T100_v2.mp4  # RGB-ID video
  +-- bkg.bmp                   # background image
  +-- alpha10-4_T100_v2.feather  # robot controller data
```

The two videos must belong to the same experiment. By default, the postprocessing configuration expects the RGB video to have `id_` added before the dynamics-video name, the background to be called `bkg.bmp`, and the Feather file to have the experiment name. These names can be changed in `postprocessing/pipeline.yaml`.

Important. Keep the files for one experiment in the same date folder. The postprocessing script uses this layout to build paths and to avoid pairing a video with the wrong background or controller data.

4 Core analysis: one experiment

This section describes each stage separately. Running the stages manually is useful when checking a new experiment or debugging a configuration. For routine analyses, the config-driven workflow in Section 7 is more convenient.

4.1 Launching standard tracking

The main command analyses one video and writes the trajectory CSV given by `-output`:

```
source .venv/bin/activate

python -m main \\  
  --video data/18-09-26/alpha10-4_T100_v2.mp4 \\  
  --background data/18-09-26/bkg.bmp \\  
  --output results/18-09-26/alpha10-4_T100_v2/alpha10-4_T100_v2.csv \\  
  --config config/default.yaml
```

For a phototaxis experiment, the default configuration uses the dark/light arena split and the ROI-based detection path. The standard CSV contains:

```
time, x, y, theta, particle
```

Here `particle` is TrackPy's temporary identity. It is not the physical robot ID assigned to the robot in the experiment; that mapping is made later using the RGB-ID recording.

4.2 Parallel processing of one video

The dynamics video can be split into contiguous frame ranges and processed by multiple workers. This is different from running several experiments in parallel: here, several processes work on different parts of the same video.

For normal phototaxis tracking, independent frame chunks can be processed by multiple workers:

```
python -m main \\  
  --video data/18-09-26/alpha10-4_T100_v2.mp4 \\  
  --background data/18-09-26/bkg.bmp \\  
  --output results/18-09-26/alpha10-4_T100_v2/alpha10-4_T100_v2.csv \\  
  --config config/default.yaml \\  
  --workers 2 \\  
  --warmup-frames 50
```

The video is split into contiguous frame ranges. Each worker receives context frames before its assigned range so that the phototaxis ROI detector can initialise its local state. Only the requested range is retained from each worker. The parent process then restores frame order and performs one global TrackPy linking pass, allowing temporary identities to continue across chunk boundaries.

The number of context frames is controlled by `-warmup-frames`; 50 is a reasonable starting value. Increase it if robots can move a long distance near a chunk boundary. The warm-up frames are processed again by neighbouring workers, so a small amount of redundant work is expected.

Multiprocessing has the following restrictions:

- it is intended for standard dynamics tracking, including phototaxis;
- it cannot be combined with `RGB_ID_ANALYSIS: true`;
- it cannot be combined with `-visualize`;
- all workers must write to the same final output only through the parent process; do not start two jobs with the same output path;
- two workers are a sensible first benchmark on a laptop. More workers do not necessarily mean proportionally greater speed, because video decoding, memory bandwidth, and circle detection can become bottlenecks.

After a parallel run, compare its processing time and output with a sequential run on a representative video. In particular, check the number of rows, the number of detected robots per frame, and the continuity of trajectories around chunk boundaries.

4.3 RGB-ID tracking

The RGB-ID video is processed with `RGB_ID_ANALYSIS: true`. In normal use, `run_pipeline.py` creates a temporary configuration, sets the expected robot count, and launches this step. For a direct run, first create a copy of the tracking configuration and enable RGB-ID analysis in that copy:

```
cp config/default.yaml config/rgb_id_experiment.yaml
```

Then set `RGB_ID_ANALYSIS: true` and the correct `N_POGO` in `config/rgb_id_experiment.yaml`, and run:

```
python -m main \\  
  --video data/18-09-26/id_alpha10-4_T100_v2.mp4 \\  
  --background data/18-09-26/bkg.bmp \\  
  --output results/18-09-26/alpha10-4_T100_v2/RGB_ID.csv \\  
  --config config/rgb_id_experiment.yaml
```

The RGB-ID implementation writes `RGB_ID.csv` in the directory containing the requested output path. It contains:

```
frame, time, id, x, y, theta, led_x, led_y, R, G, B
```

The RGB values are measured around the estimated LED position. They are decoded into ternary symbols during the merging stage. RGB-ID processing is intentionally single-worker: the flashing sequence and the common robot pose must remain synchronised.

4.4 Decoding IDs and merging controller data

The merge script combines the dynamics CSV, the RGB-ID CSV, the RGB-ID calibration output, and the Feather file. It requires the delay between the dynamics and RGB-ID recordings in seconds. The RGB flashing start frame can be detected automatically when `-start-rgb-frame` is omitted.

For a manual merge, use a measured delay in place of `<seconds_difference>`:

```
python postprocessing/process_id_trits.py \\  
--rgb-id-path results/18-09-26/alpha10-4_T100_v2/RGB_ID.csv \\  
--probe-path data/18-09-26/alpha10-4_T100_v2.feather \\  
--dyn-path results/18-09-26/alpha10-4_T100_v2/alpha10-4_T100_v2.csv \\  
--seconds-difference <seconds_difference> \\  
--expected-n-robots 64 \\  
--results results/18-09-26/alpha10-4_T100_v2/merged_alpha10-4_T100_v2.csv
```

The script saves an intermediate `score_calib_pose_id.csv` next to `RGB_ID.csv` and writes the final merged dataset to the path given by `-results`. The config-driven workflow can estimate `seconds_difference` automatically and is therefore recommended for routine processing.

4.5 Generating summary slides

Once the merged CSV exists, the slide generator can be run directly:

```
python postprocessing/experiment_slides_generator.py \\  
--csv results/18-09-26/alpha10-4_T100_v2/merged_alpha10-4_T100_v2.csv \\  
--video data/18-09-26/alpha10-4_T100_v2.mp4 \\  
--controller tanh \\  
--experiment-id alpha10-4_T100_v2 \\  
--output results/18-09-26/alpha10-4_T100_v2/alpha10-4_T100_v2
```

This produces a PNG and a PDF with the same output base name. Optional plotting parameters, such as the light threshold, communication time, snapshot times, and controller parameters, can be added as command-line options. The available options are also exposed through the `slides.options` section of `postprocessing/pipeline.yaml`.

5 Important configuration parameters

The main tracking settings are in `config/default.yaml`. The parameters below are the ones most likely to change when the arena or camera setup changes.

Parameter	Meaning
N_POGO	Expected number of robots. This is a central consistency constraint for detection and recovery.
FPS	Video frame rate used to convert frame numbers into seconds.
THRESHOLD	Foreground binarization threshold for non-phototaxis detection.
AREAS	Minimum and maximum accepted contour area in pixels squared.
PERIMETERS	Minimum and maximum accepted contour perimeter in pixels.
CENTER, RADIUS	Circular arena mask centre and radius in pixels.
SEARCH_RANGE	Maximum TrackPy linking distance in pixels.
MEMORY	Number of frames for which a temporarily missing robot can retain its TrackPy identity.
POGOBOT_DIAMETER_CM, PIXEL_DIAMETER	Pixel-to-centimetre calibration values.
PHOTOTAXIS_ANALYSIS	Enables the dark/light split processing path.
PHOTOTAXIS_FAST	Enables cached masks and ROI-based detection after initialization.
PHOTOTAXIS_THRESHOLDS	Dark and light segmentation thresholds.
HOUGH_CIRCLES	Circle-detection parameters used to identify robot bodies.
RGB_ID_ANALYSIS	Selects RGB-ID processing instead of standard trajectory processing.

In practice, start with the arena geometry, robot count, foreground thresholds, phototaxis thresholds, and calibration values. Change these in YAML rather than editing the detection code. The postprocessing pipeline creates a temporary copy of the tracking configuration for each experiment and overwrites N_POGO and RGB_ID_ANALYSIS there, so the shared default file is not modified.

Attention. It is assumed that the number of robots is fixed for an experiment. The detector uses the N_POGO parameter when checking and recovering incomplete frames.

6 Relevant modules and classes

6.1 Core tracking

- `main.py`: parses the command line, constructs `VideoProcessor`, and selects standard, parallel, or RGB-ID processing.
- `video_processing.py` in `src/`: contains the `VideoProcessor` class. Its principal methods are `process()`, `process_parallel()`, and `process_rgb_id()`.
- `src/utils.py`: contains image preprocessing, contour and circle detection, pose estimation, pixel conversion, and tracking utilities.
- `src/plot_helpers.py`: contains frame debugging, trajectory plotting, RGB-ID debug views, and GIF/contact-sheet helpers.

The standard path subtracts the background, applies the arena mask, detects robot candidates, estimates their orientation, and stores one row per robot and frame. TrackPy then links the temporary particles over time.

6.2 Postprocessing modules

- `postprocessing/pipeline.yaml`: editable experiment list, paths, robot counts, merge settings, and slide settings.

- `postprocessing/run_pipeline.py`: runs tracking, merging, and slide generation from that configuration.
- `postprocessing/process_id_trits.py`: detects the RGB flashing start, aggregates RGB windows, decodes ternary IDs, assigns real robot IDs, and merges controller data.
- `postprocessing/experiment_slides_generator.py`: creates PNG and PDF summary figures for step, Gaussian, and tanh controllers.
- `postprocessing/batch_analysis_tmux.sh`: optional interactive tmux launcher for independent experiment workers.

7 Run all analysis all at once

7.1 Configure an experiment

Edit `postprocessing/pipeline.yaml` before starting a new analysis. An entry gives the date folder, dynamics video, robot count, optional per-video worker count, and any file-name overrides:

```
experiments:
- id: alpha10-4_T100_v2
  date: 18-09-26
  video: alpha10-4_T100_v2.mp4
  n_robots: 64
  workers: 2
  probe_file: alpha10-4_T100_v2.feather
  outputs:
    normal: "{id}.csv"
    rgb_id: "RGB_ID.csv"
    merged: "merged_{id}.csv"
    slides: "{id}"
  merge:
    seconds_difference: auto
    start_rgb_frame: auto
  slides:
    enabled: true
    controller: tanh
```

With `auto`, the merge step analyses the configured initial interval of the dynamics CSV for the first sustained population movement. It also searches the RGB-ID CSV for the LED onset. Replace either value with a numeric value if the automatic estimate is not suitable for a particular experiment. The optional `merge.auto` block controls the motion-detection thresholds.

The two worker settings have different meanings:

- `workers` belongs to one experiment and controls how many processes analyse its dynamics video. `workers: 1` keeps the sequential implementation; `workers: 2` splits one video into two chunks.
- `batch.max_workers` is global and controls how many experiments are launched concurrently by `run_pipeline.py`.

Their product is a useful upper bound on the number of video-processing workers. For example, `batch.max_workers: 2` and `workers: 2` may use up to four dynamics workers at once. Start conservatively, especially on a laptop, and increase the values only after benchmarking.

7.2 Run all stages

From the repository root, with the virtual environment active:

```
python postprocessing/run_pipeline.py
```

The script runs the stages in this order:

1. standard and RGB-ID tracking for each experiment;
2. RGB-ID decoding and merging with the Feather data;
3. PNG and PDF slide generation.

Different experiments run concurrently as independent subprocesses according to `batch.max_workers`. Within each experiment, the dynamics video can additionally use its own `workers` value. A temporary tracking configuration is created for each experiment, so `config/default.yaml` is left unchanged.

For a first complete test, it is sensible to use one experiment and conservative settings:

```
batch:
  enabled: true
  max_workers: 1

experiments:
  - id: alpha10-4_T100_v2
    date: 18-09-26
    video: alpha10-4_T100_v2.mp4
    n_robots: 64
    workers: 2
```

Once this completes successfully, additional experiments can be enabled and `batch.max_workers` can be increased.

7.3 Run one stage or one experiment

```
# Standard and RGB-ID tracking only
python postprocessing/run_pipeline.py --stage batch

# RGB-ID tracking only (standard tracking CSV must already exist)
python postprocessing/run_pipeline.py --stage rgb

# Merge existing tracking outputs
python postprocessing/run_pipeline.py --stage merge

# Generate slides from existing merged outputs
python postprocessing/run_pipeline.py --stage slides

# Restrict any stage to one configured experiment
python postprocessing/run_pipeline.py \
  --stage merge \
  --experiment alpha10-4_T100_v2

# Inspect commands without executing them
python postprocessing/run_pipeline.py --dry-run
```

The `-stage batch` command runs standard dynamics tracking followed by RGB-ID tracking. It does not perform merging or slide generation. The `-stage rgb` command is useful when the dynamics CSV already exists and only the RGB-ID video needs to be reprocessed.

The output for one experiment is stored under:

```
results/<date>/<experiment>/
+-- <experiment>.csv           # dynamics tracking output
+-- RGB_ID.csv
+-- score_calib_pose_id.csv    # RGB-ID decoding intermediate
+-- merged_<experiment>.csv   # final merged dataset
+-- <experiment>.png          # summary figure
+-- <experiment>.pdf          # summary figure
+-- tracking.log
+-- rgb_tracking.log
+-- merge.log
+-- slides.log
```

8 Understanding the merged data and slides

The merged dataset contains the main quantities used for swarm analysis:

`t, x, y, theta, q, w, id`

Here `t` is time after the detected offset, `x` and `y` are robot coordinates, `theta` is orientation, `q` and `w` are controller variables from the Feather file, and `id` is the decoded robot identifier. The exact time origin depends on the delay supplied to the merge step.

The slide generator includes:

- the controller response panel;
- representative video frames at the initial, middle, and final times;
- the fraction of robots in the light region;
- per-robot and mean score/weight traces;
- relative density, weight, and score space-time fields.

The controller type is selected in the postprocessing configuration:

```
slides:
  enabled: true
  controller: tanh      # step, gaussian, or tanh
```

Additional slide options, such as light threshold, communication time, snapshot times, bin counts, and controller parameters, can be supplied under `slides.options`.

9 Troubleshooting checklist

If an analysis fails or the result looks wrong, check the following:

1. Confirm that the active interpreter is the project virtual environment and Python is at least 3.11.
2. Confirm that the dynamics video, `id_ RGB` video, background image, and Feather file belong to the same experiment.
3. Confirm that `N_POGO` or `n_robots` matches the number of robots actually recorded.
4. Check arena centre, radius, foreground thresholds, and phototaxis dark/light thresholds.
5. Inspect `tracking.log`, `rgb_tracking.log`, and the terminal output before changing algorithm code.
6. Run `-dry-run` to verify all generated paths and commands.
7. If automatic timing detection is unsuitable for one experiment, replace one or both `auto` values with manual values in `pipeline.yaml`.
8. If parallel and sequential runs differ near a chunk boundary, increase `-warmup-frames` for the direct command and compare the resulting trajectories again.
9. If the computer becomes unresponsive during a batch run, reduce either `batch.max_workers` or the per-experiment `workers` value.

Important. Keep the raw input data unchanged. Write generated CSV files, logs, and figures under `results/`; keep reusable settings in YAML files.

10 Final remarks

Pogotrack is a shared research tool and will continue to evolve as new Pogobots experiments are added. If you find a bug or improve the analysis, please open an issue or submit a pull request. Hope it helped :)